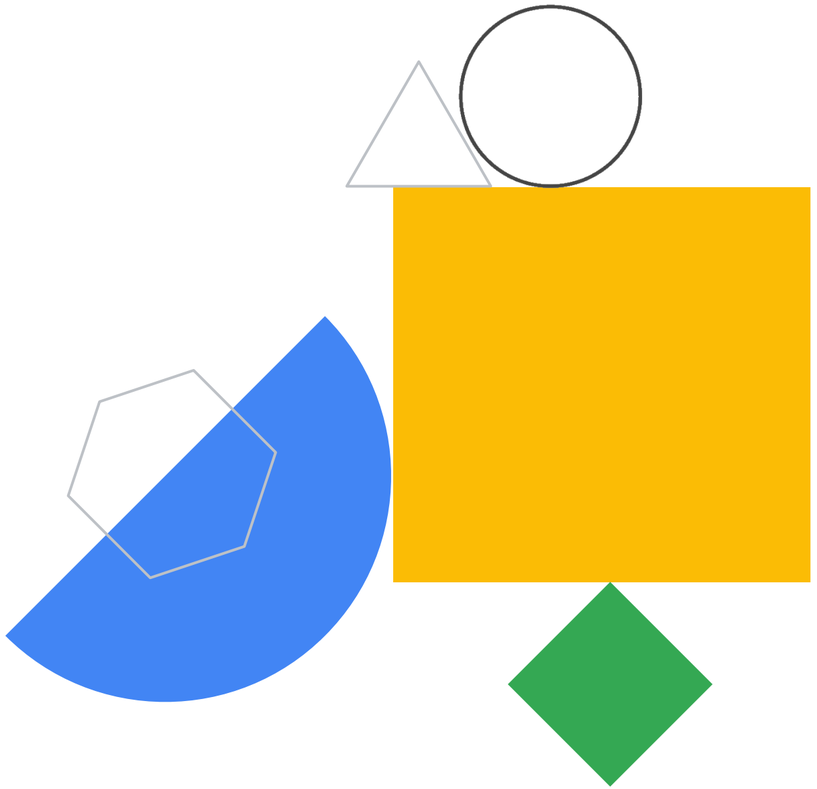
Google Cloud

Configuring Google Cloud Services for Observability



Proprietary + Confidential

Objectives

01 Use Ops Agent with Compute Engine.

02 Explain the benefits of using Google Cloud Managed Service for Prometheus.

03 Explain the usage of Prometheus Query Language (or PromQL) to query Cloud Monitoring metrics.

04 Explain custom metrics.



Proprietary + Confidential

In this section, you explore

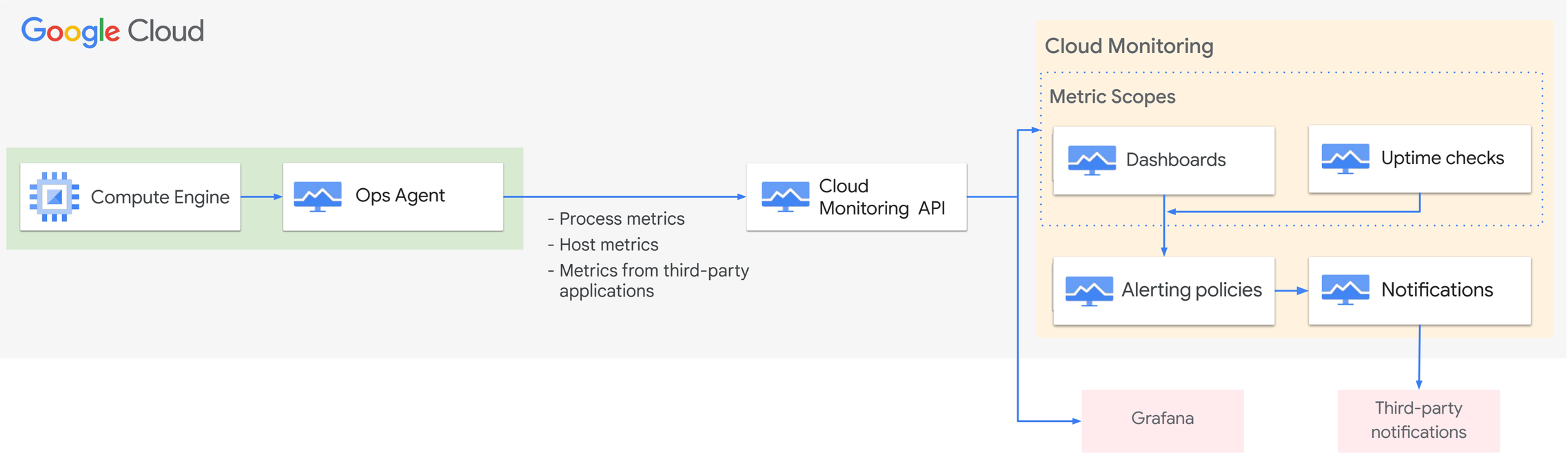


- Introduction to Ops Agent
- Non-VM resources
- Cloud Operations for GKE
- Google Cloud Managed Service for Prometheus
- Exposing user-defined metrics

Proprietary + Confidential

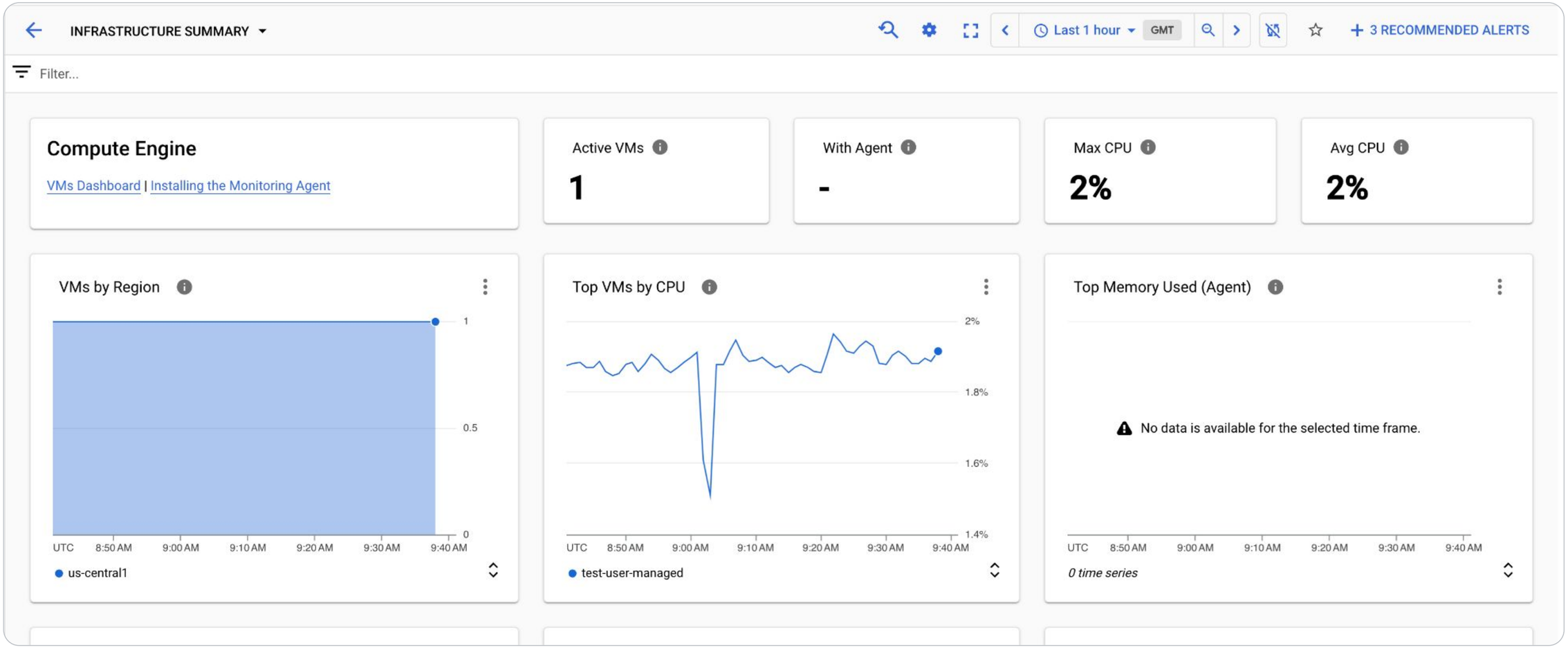
What is Ops Agent?

Ops Agent gathers system and application metrics from **VM instances** and sends them to Monitoring.



Why Ops Agent?

Proprietary + Confidential



Google Cloud

Proprietary + Confidential

Why Ops Agent?

Benefits of running the Ops Agent inside the VM:

- Monitors your VM instances without the need for any additional configuration after the installation.
- Monitors third-party applications and supports both Windows and Linux guest OS.
- Exposes many additional process metrics beyond memory and gives you better visibility to CPU, disk, and network performance.
- Exposes metrics beyond the 80+ metrics that Cloud Monitoring already supports for Compute Engine.
- Unifies gathering of metrics and logs into a single agent.
- Ingest user-defined metrics in Prometheus format.

Proprietary + Confidential

Three ways to install Ops Agent

Single VM	Install agents on a single VM by using command line.
Using policies	Install agents on a fleet of VMs by using agent policies.
Using automation tools	Install agents on a fleet of VMs by using automation tools.

Proprietary + Confidential

Installing the Ops Agent by using the agent policy

Step 1 **Install** the beta component of the gcloud CLI

Step 2 Enable the APIs and set the proper permissions for using the Google Cloud CLI by running the script: **set-permissions.sh**

Step 3 Create a policy.

```
gcloud beta compute instances ops-agents policies create ops-agents-test-policy
--agent-rules="type=ops,enable-autoupgrade=false;type=metrics,enable-autoupgrade=false"
--description="A test policy." --os-types=short-name=centos,version=7
--instances=zones/us-central1-a/instances/test-instance
```

Proprietary + Confidential

Installing the Ops Agent on individual VMs

Step 1	Click the name of the VM that you want to install the agent on.
Step 2	Click the **Observability** tab.
Step 3	Click **Install Ops Agent**.
Step 4	Click **Run in Cloud Shell**.
Step 5	Press **Enter** to run the command and click **Authorize** to allow Cloud Shell to install the agent.

Ensure that the Logging API is enabled and that the Logs Writer role is configured for the service account serving the Ops agent.

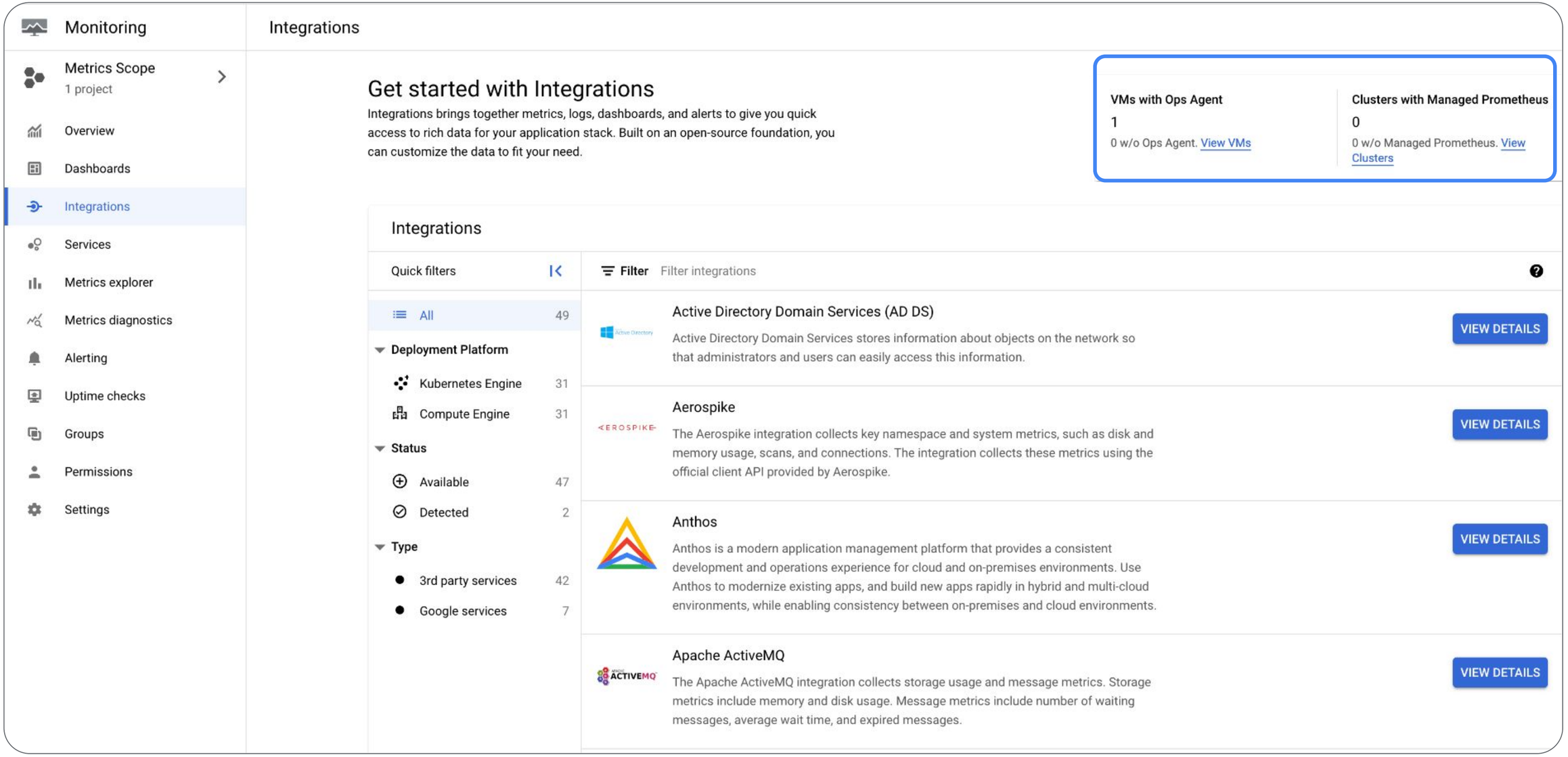
Proprietary + Confidential

Troubleshooting: Agent is not sending metrics to Cloud Monitoring

1. Check the metrics module in syslog.
2. If there are no logs, then the agent service is not running.
3. If you see permission_denied errors when writing to the Monitoring API, enable the Monitoring API and the Monitoring metric writer role.

Integrations page

Proprietary + Confidential



Proprietary + Confidential

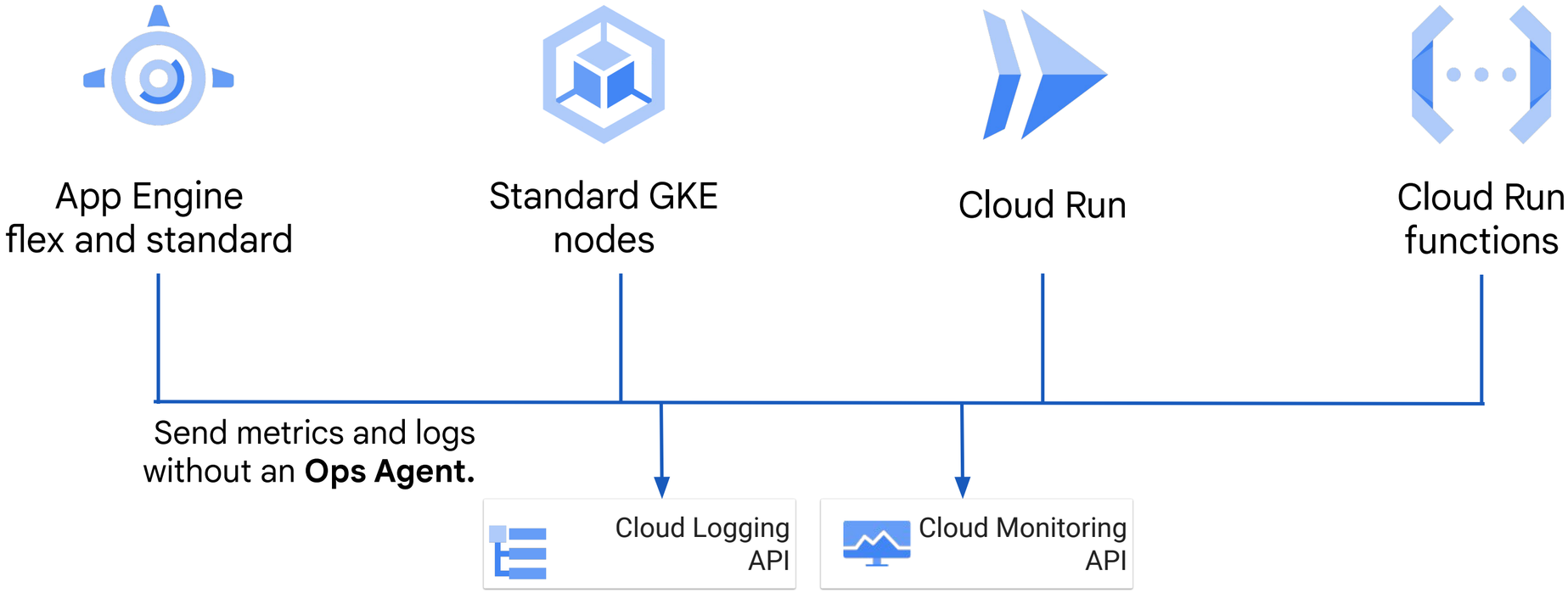
In this section, you explore



- Introduction to Ops Agent
- Non-VM Resources
- Cloud Operations for GKE
- Google Cloud Managed Service for Prometheus
- Exposing user-defined metrics

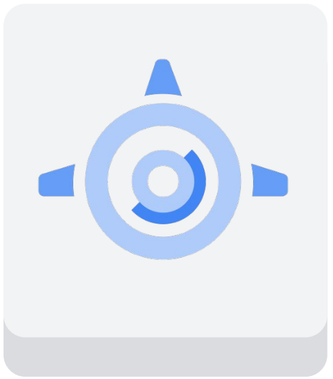
Proprietary + Confidential

Services with built-in logging and monitoring support



Proprietary + Confidential

App Engine



App Engine

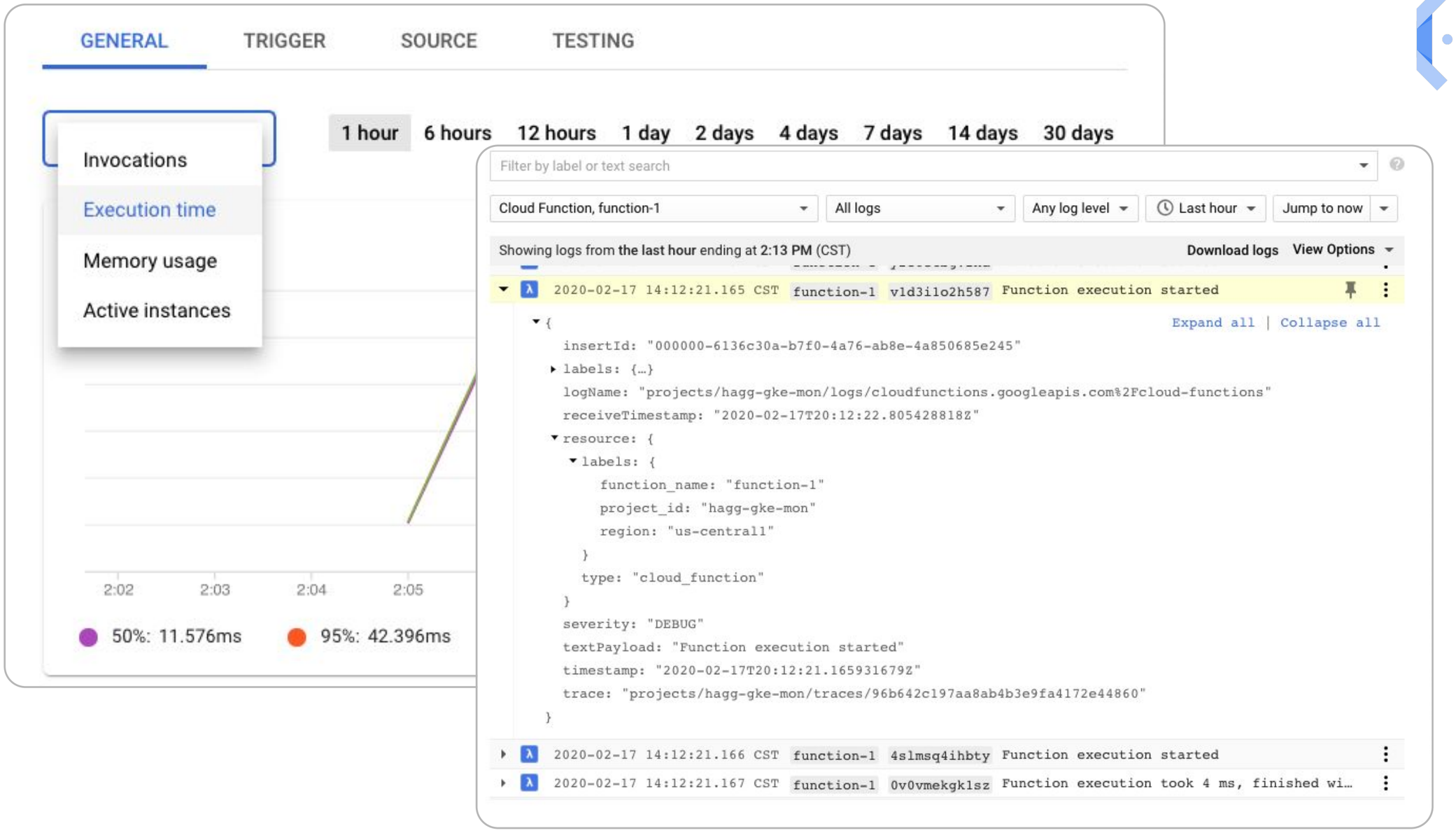
Standard and flexible environments support Monitoring.

Standard and flexible environments support Logging.

- Support writing to stdout or stderr from code.
- Support language-specific logging APIs (like Winston on Node.js).
- Let you view logs under GAE Application resource.

Monitoring and logging in Cloud Run functions

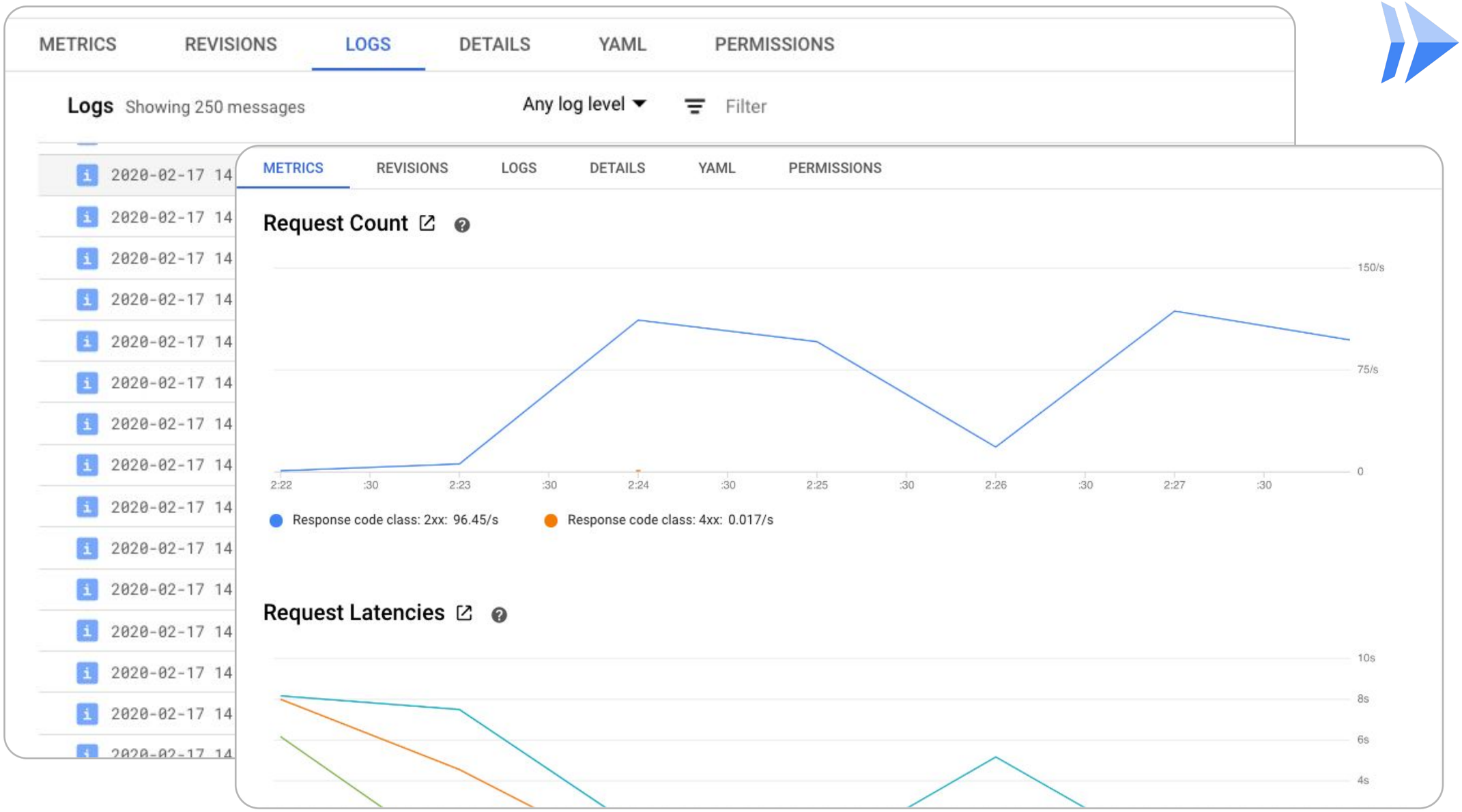
Proprietary + Confidential



Google Cloud

Monitoring and logging in Cloud Run

Proprietary + Confidential



Proprietary + Confidential

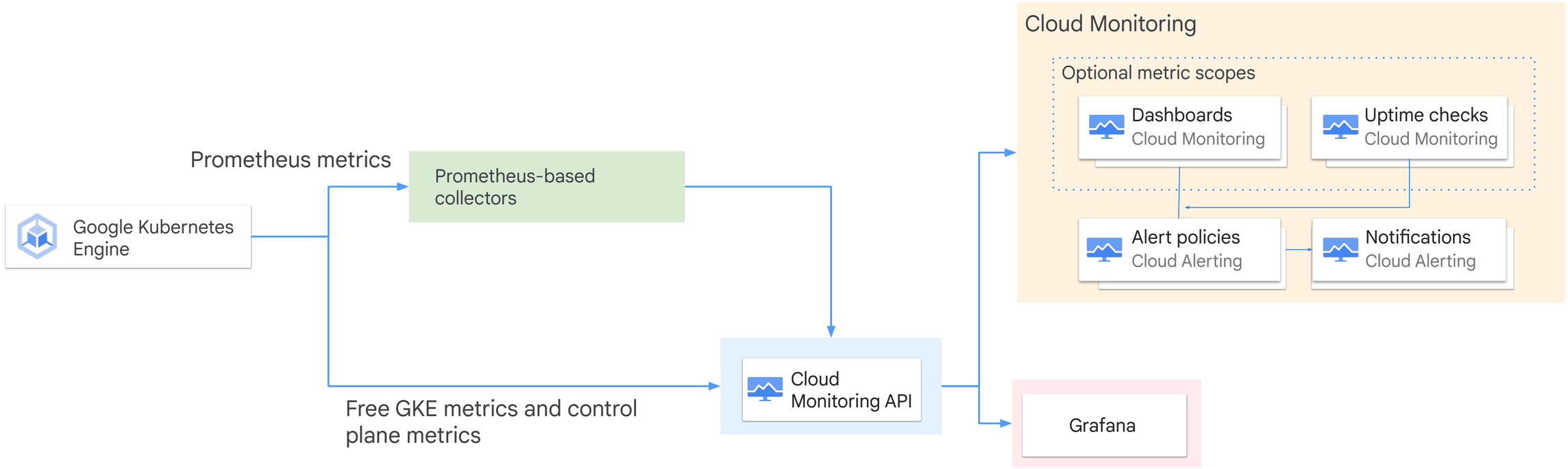
In this section, you explore



- Introduction to Ops Agent
- Non-VM Resources
- Cloud Operations for GKE
- Google Cloud Managed Service for Prometheus
- Exposing user-defined metrics

Proprietary + Confidential

Cloud Monitoring for Google Kubernetes Engine



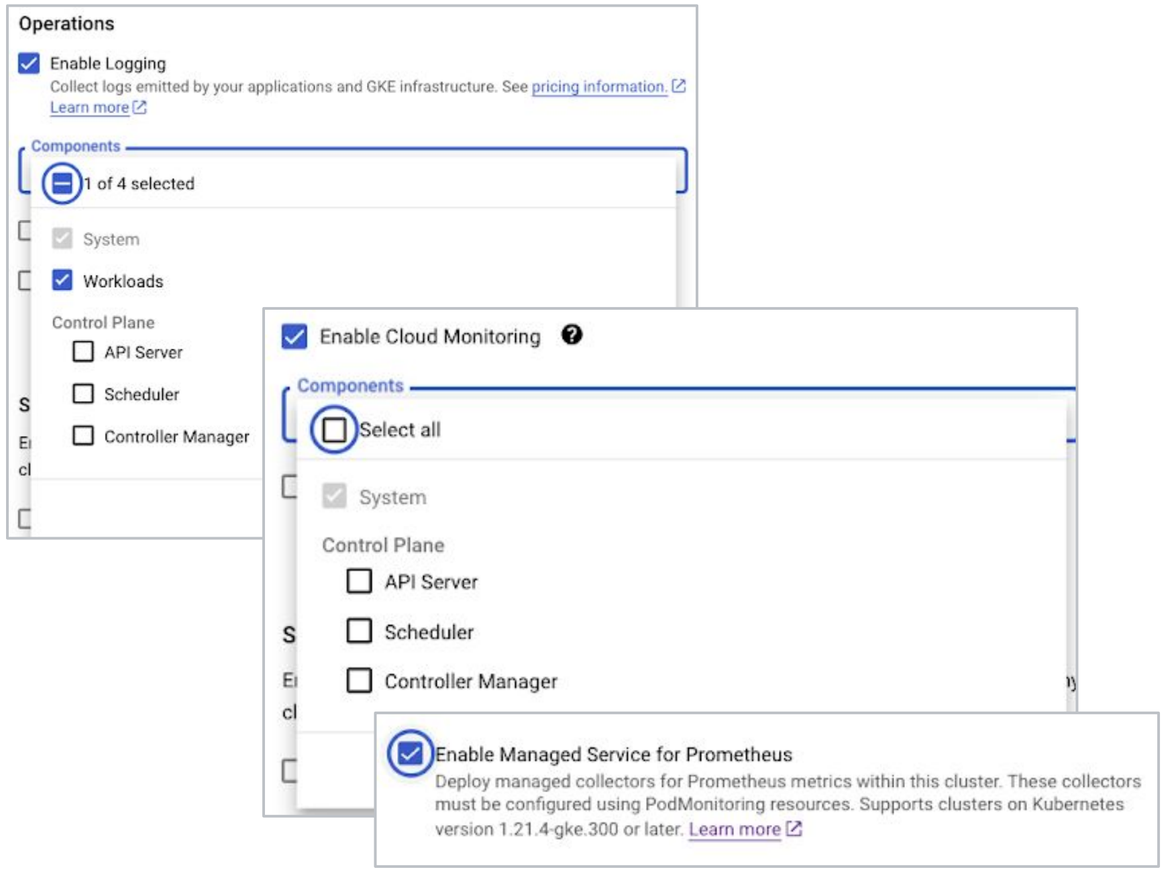
GKE includes integration with:

- Cloud Logging and Cloud Monitoring
- Google Cloud Managed Service for Prometheus

Configuring Cloud Operations for GKE clusters

For new cluster

- Cloud Monitoring and Cloud Logging is enabled by default.
- Under **Components**, select the components from which you want to collect logs and metrics.
- Optionally, you can enable Managed Service for Prometheus.

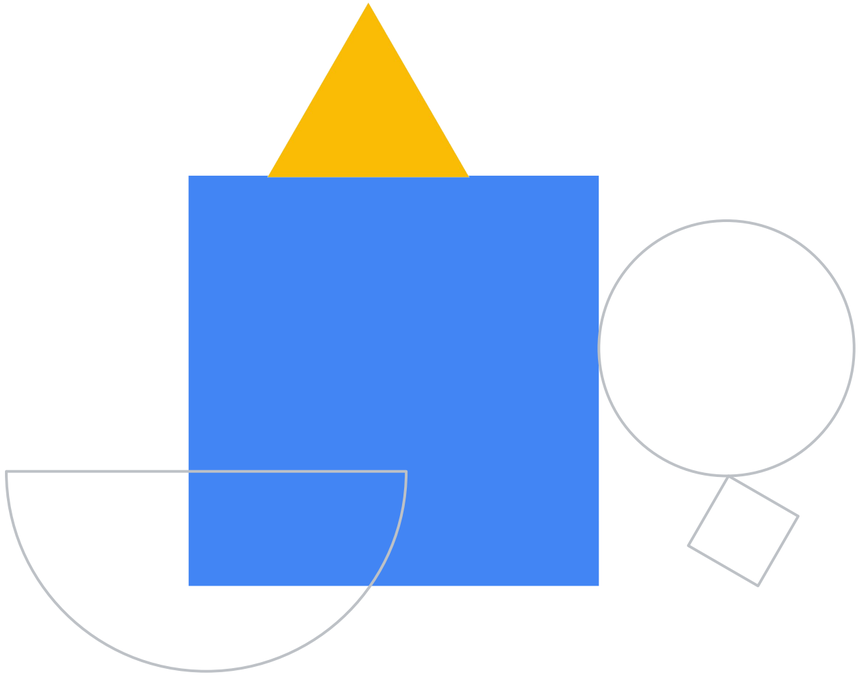


Proprietary + Confidential

Demo

GKE Dashboards

- Deploy a sample application
- Navigate the out-of-the-box GKE dashboards to view operational data.
- Configuring the dashboard tables.



Proprietary + Confidential

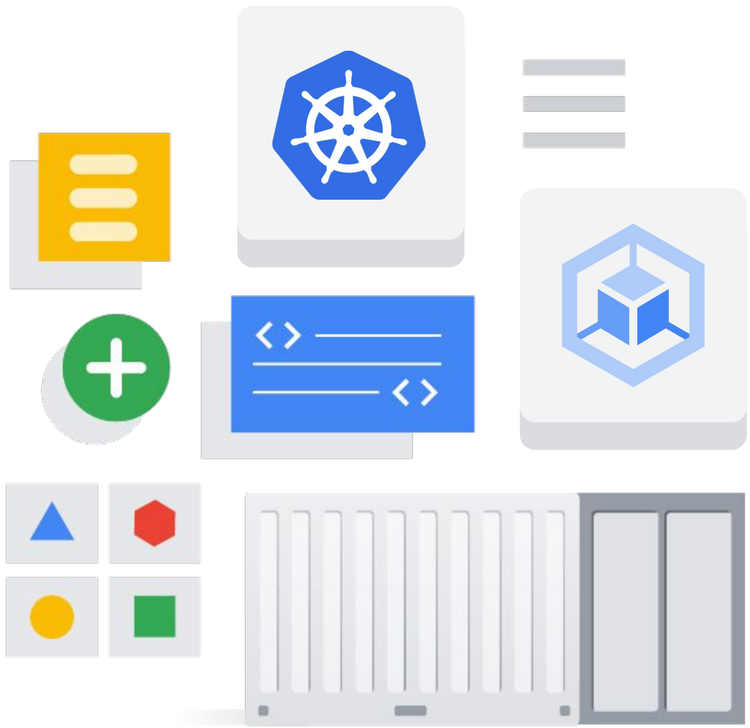
In this section, you explore



- Introduction to Ops Agent
- Non-VM Resources
- Cloud Operations for GKE
- Google Cloud Managed Service for Prometheus
- Exposing user-defined metrics

Proprietary + Confidential

Google Cloud Managed Service for Prometheus



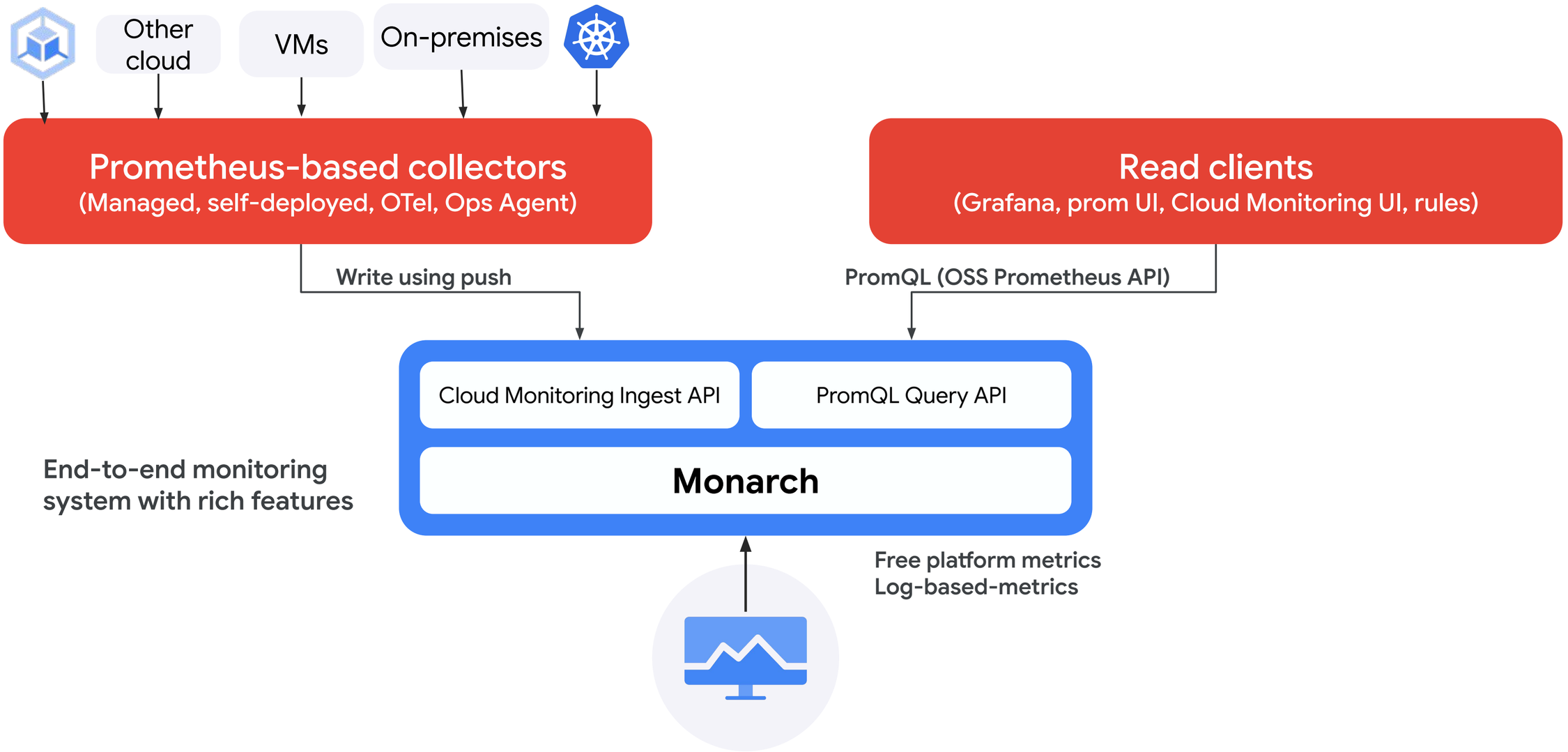
Collects, stores, and analyzes Prometheus metrics.

Collects metrics from both GKE and Compute Engine environments.

Helps make GKE easier to use with maintained metric collectors.

Proprietary + Confidential

The Managed Service for Prometheus ecosystem



Proprietary + Confidential

Data collection options

Managed data collection

- Managed data collection in Kubernetes environments
- Recommended approach for all Kubernetes environments

Self-deployed collection

- Prometheus installation managed by the user
- Suitable for quick integrations into complex environment

The Ops Agent

- Prometheus metrics scraped and sent by the Ops Agent
- Recommended for sending Prometheus metric data

OpenTelemetry

- Deployed in any compute or Kubernetes environment
- Recommended for cross-signal workflows such as exemplars

Proprietary + Confidential

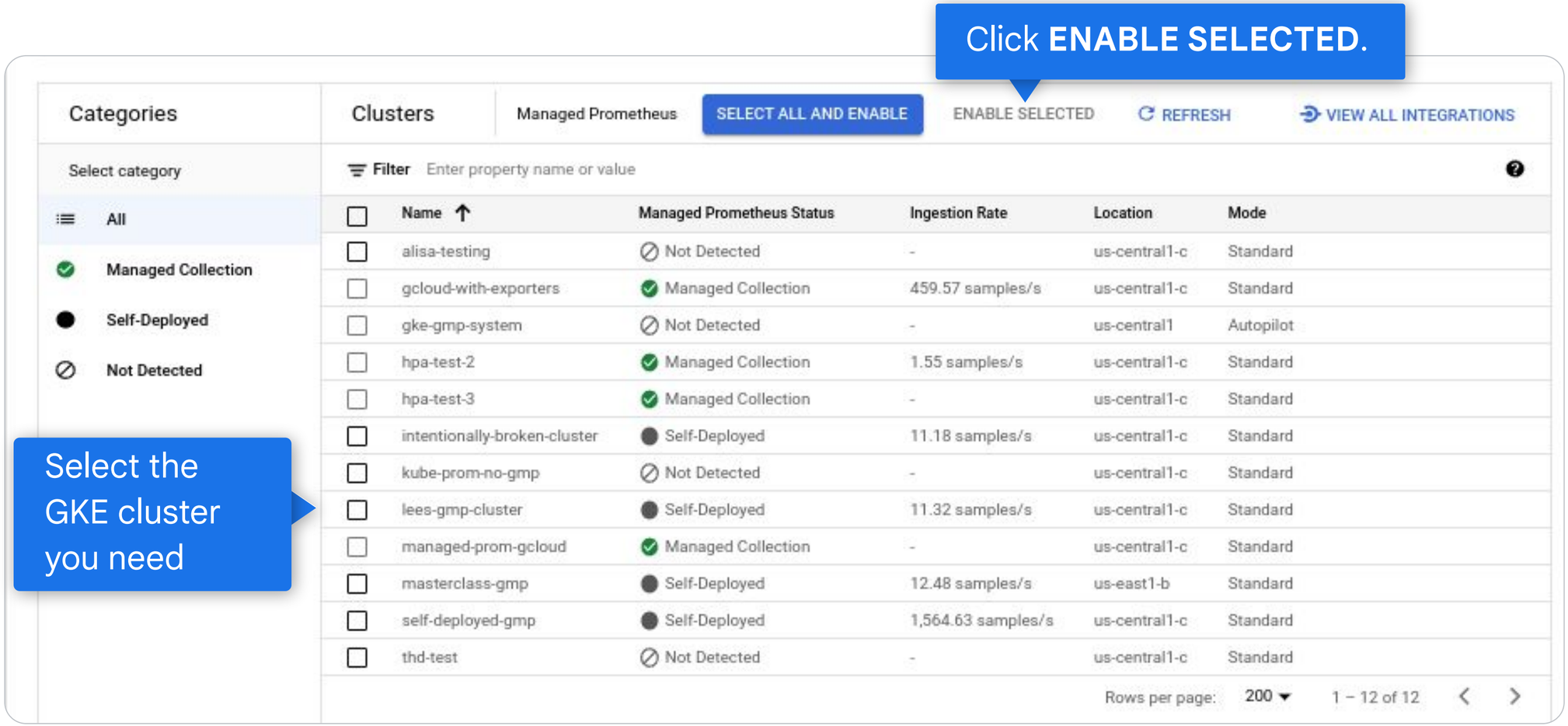
Rule and alert evaluation

Managed Service for Prometheus provides a standalone rule evaluator for evaluating recording and alerting rules.

- There’s no need to co-locate the data because rules are evaluated against a multi-project metrics scope.
- Migration to managed service for Prometheus is made easier with standard Prometheus rule files format.

Proprietary + Confidential

Setting up managed collection for GKE clusters



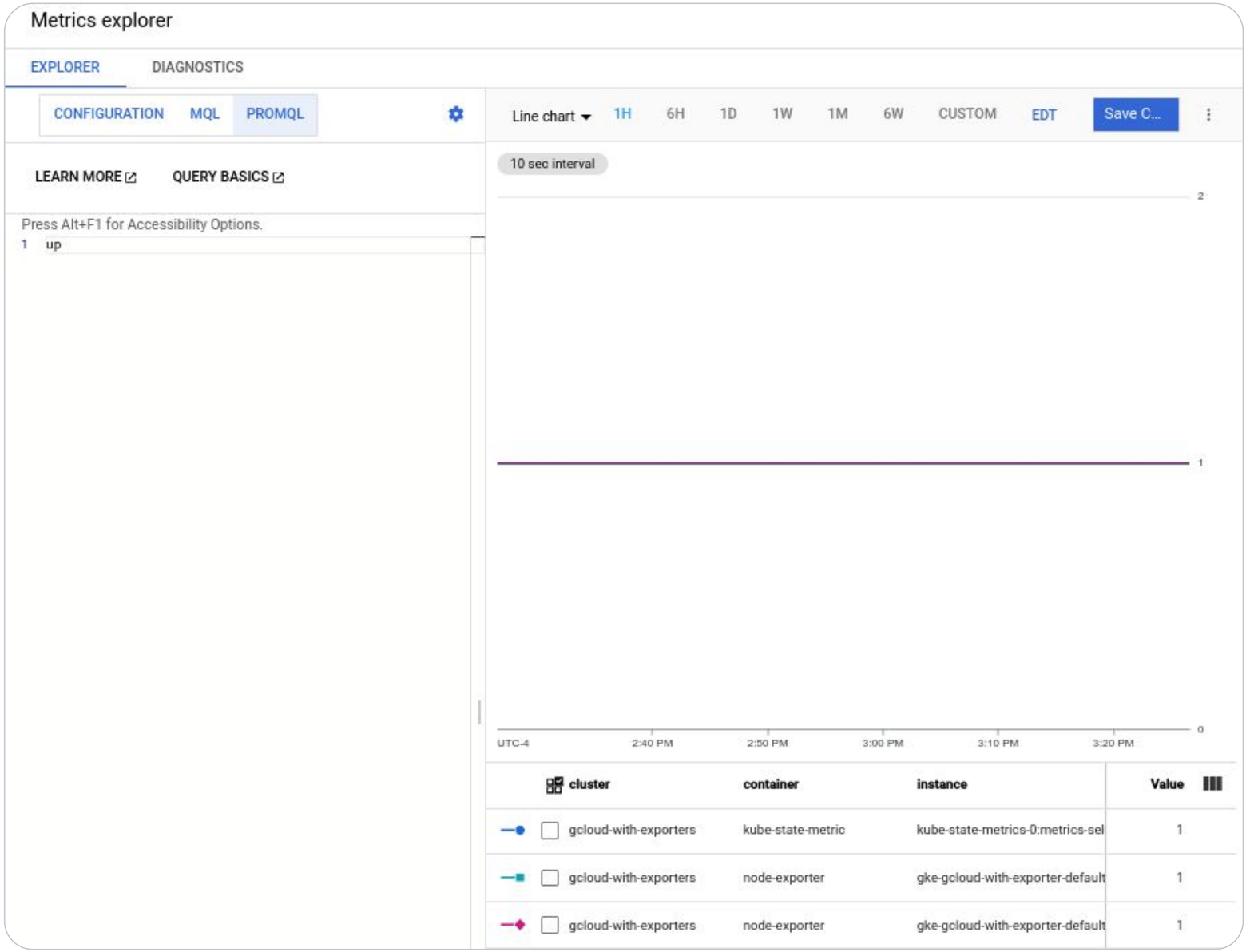
Proprietary + Confidential

Deploy a PodMonitoring resource to scrape metrics

```
apiVersion: monitoring.googleapis.com/v1
kind: PodMonitoring //PodMonitoring resource
metadata:
  name: prom-example
spec:
  selector:
    matchLabels:
      app: prom-example //find pods that matches the label and value
  endpoints:
  - port: metrics
     interval: 30s //scrape on a prot every 30 seconds
```

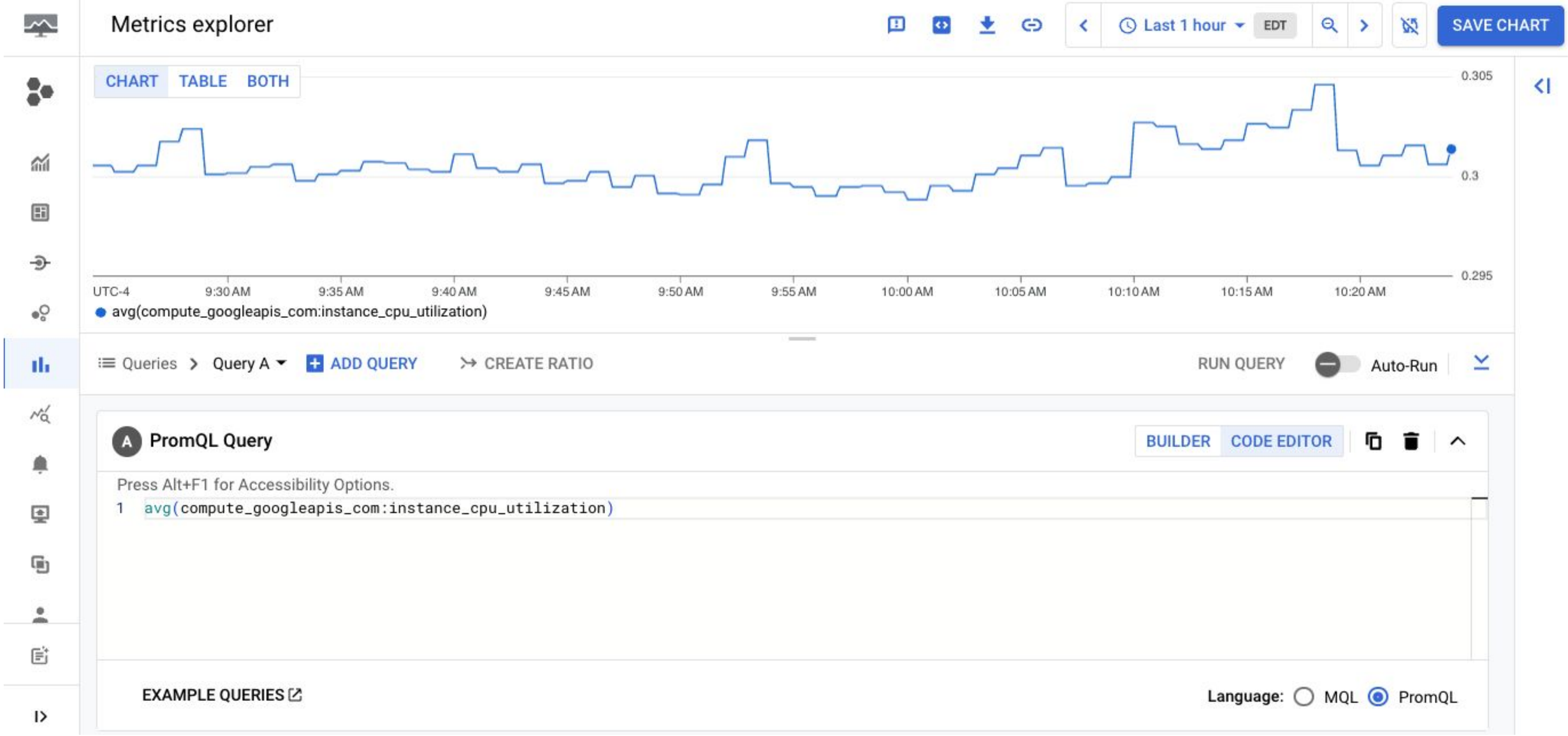
```
kubectl -n NAMESPACE_NAME apply -f <pod-monitoring.yaml>
```

Using PromQL



PromQL example

Proprietary + Confidential



Google Cloud

Proprietary + Confidential

In this section, you explore



- Introduction to Ops Agent
- Non-VM Resources
- Cloud Operations for GKE
- Google Cloud Managed Service for Prometheus
- Exposing user-defined metrics

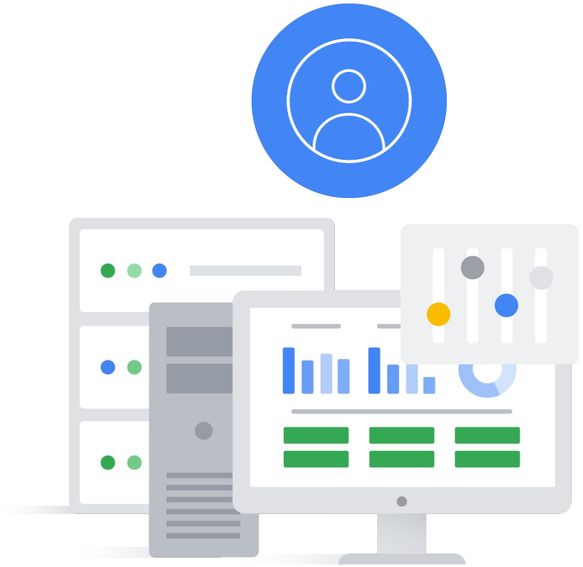
Proprietary + Confidential

User-defined metrics

- Any metrics not defined by Google Cloud are user-defined metrics.
- They are used to extract metrics that are not captured by the built-in metrics.

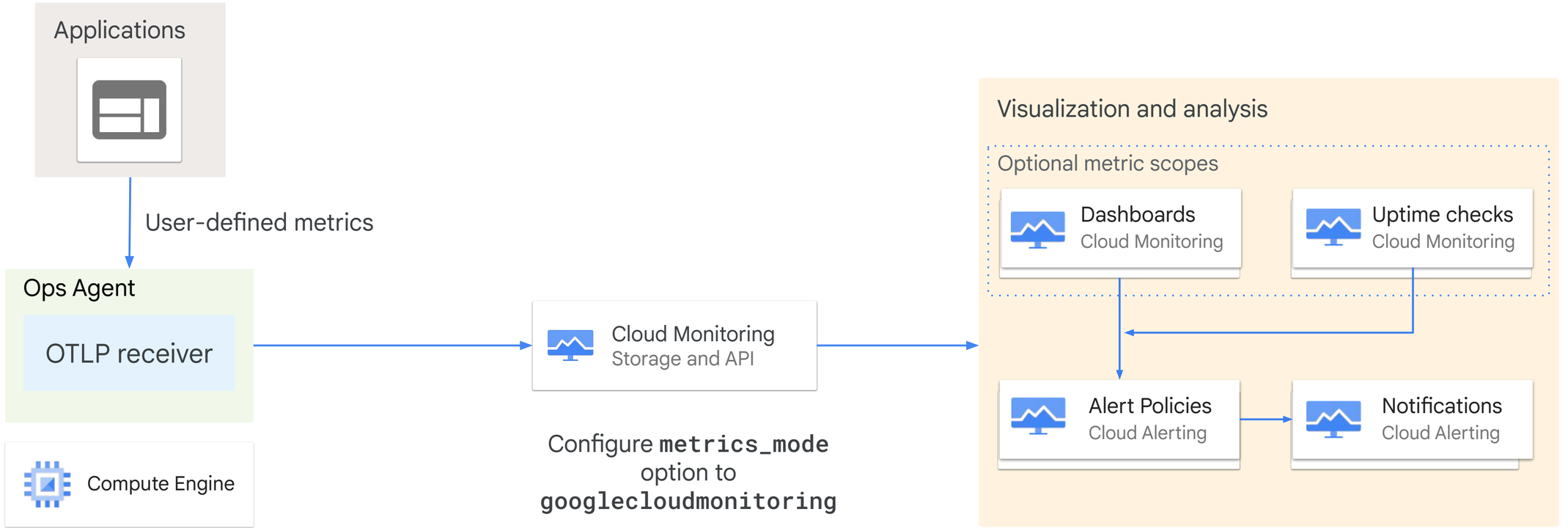
Two approaches to create:

- Use OpenTelemetry Protocol (Recommended).
- Use the Cloud Monitoring API.



Proprietary + Confidential

Collect user-defined metrics by using OTLP receiver



Proprietary + Confidential

Create user-defined metrics by using the API

```
//Custom metric descriptor example in Python
from google.cloud import monitoring_v3

client = monitoring_v3.MetricServiceClient()
project_name = client.project_path(project_id)

descriptor = monitoring_v3.types.MetricDescriptor()
descriptor.type = ('custom.googleapis.com/my_metric')
descriptor.metric_kind = (monitoring_v3.enums.MetricDescriptor.MetricKind.GAUGE)

descriptor.value_type = (monitoring_v3.enums.MetricDescriptor.ValueType.DOUBLE)

descriptor.description = 'Custom metric example.'

client.create_metric_descriptor(project_name, descriptor)
```

Proprietary + Confidential

Writing user-defined metrics

```
client = monitoring_v3.MetricServiceClient()
project_name = client.project_path(project_id)

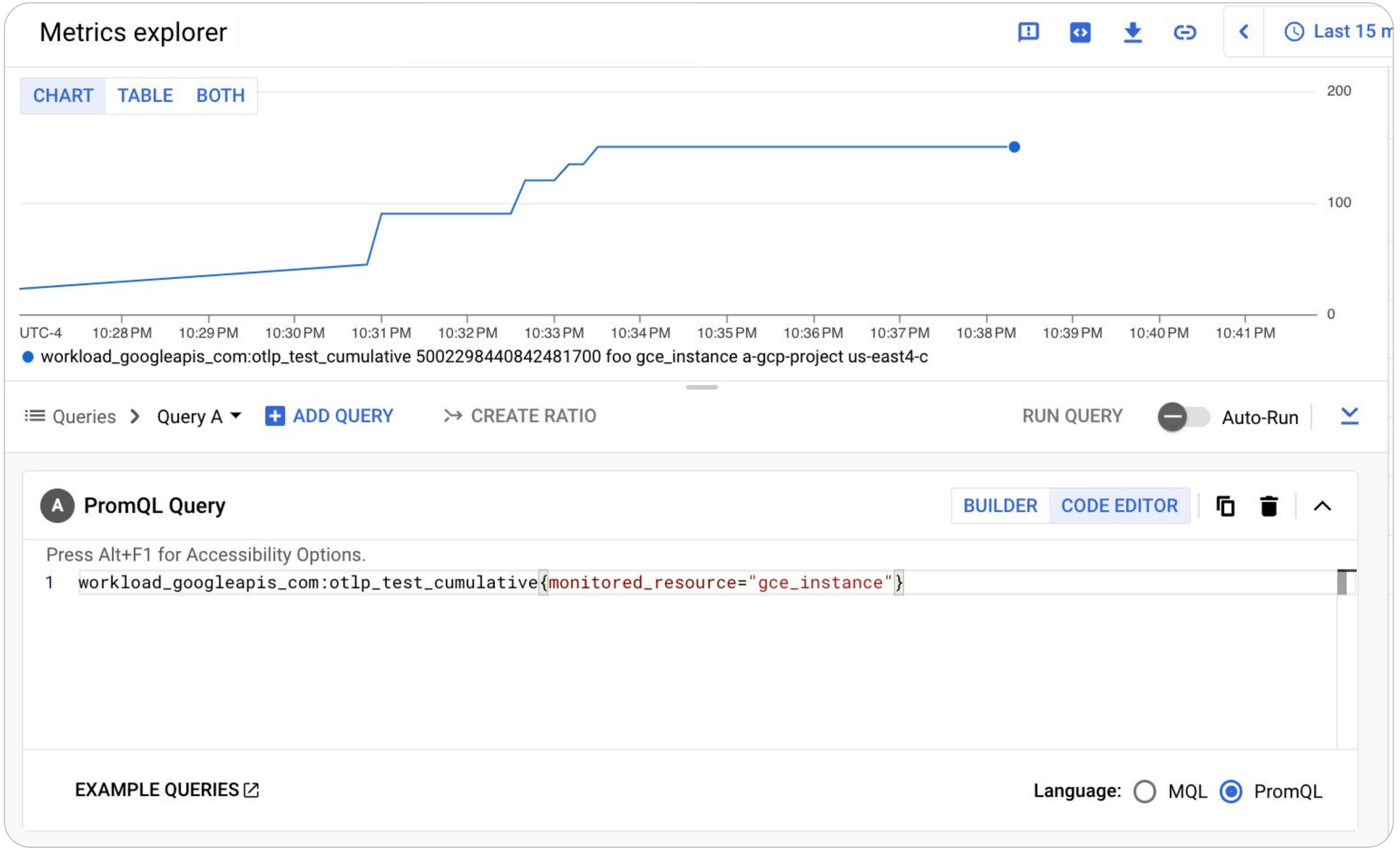
//my_metric is liked to the specified Compute Engine instance
series = monitoring_v3.types.TimeSeries()
series.metric.type = ('custom.googleapis.com/my_metric’)
series.resource.type = 'gce_instance'
series.resource.labels['instance_id']='1267890123456789'
series.resource.labels['zone'] = 'us-east4-c'

//Create point by adding to the series
point = series.points.add()
point.value.double_value = 3.14
now = time.time()
point.interval.end_time.seconds = int(now)

//Passing a list of timeseries objects to create_time_series
client.create_time_series(project_name, [series])
```

Query custom metrics

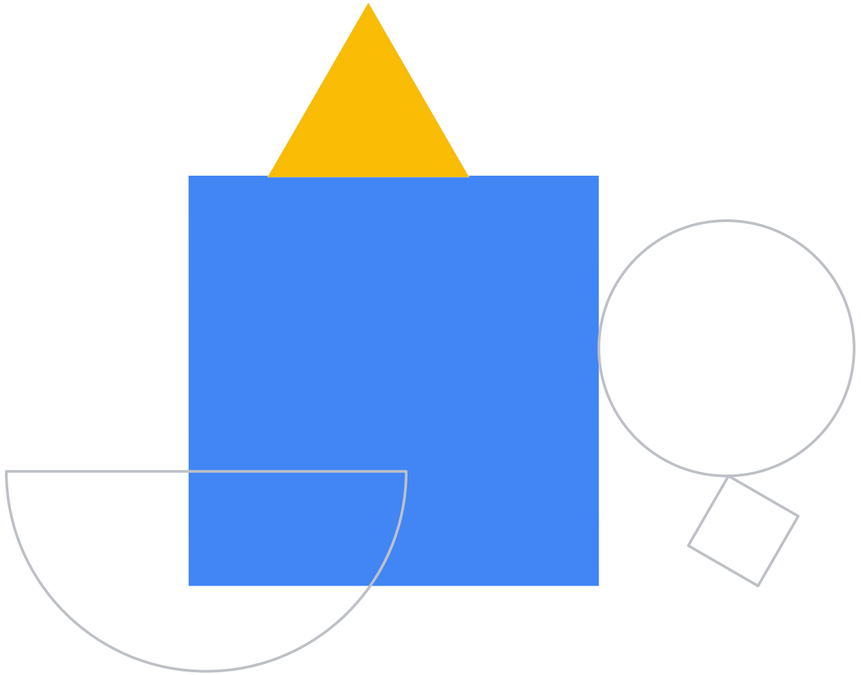
Proprietary + Confidential



Proprietary + Confidential

Lab Intro

Monitoring a Compute Engine by using Ops Agent



Proprietary + Confidential

Knowledge Check



Proprietary + Confidential

Quiz | Question 01

Question

Management wants to see an analysis of resources divided by development team, department, cost center, and application status. What could you do to make this easier?

A. Add appropriate labels to your Google Cloud resources.

B. Add appropriate tags to your Google Cloud resources.

C. Use standardized prefixes on the names of all resources.

D. Use customized logging messages that include appropriate resource metadata.

Proprietary + Confidential

Quiz | Question 01

Answer

Management wants to see analysis of resources by development team, department, cost center, and application status. What could you do to make this easier?

A. Add appropriate labels to your Google Cloud resources.



B. Add appropriate tags to your Google Cloud resources.

C. Use standardized prefixes on the names of all resources.

D. Use customized logging messages that include appropriate resource metadata.

Proprietary + Confidential

Quiz | Question 02

Question

What are the three ways to install the Ops Agent?

A. On a single VM

B. On a fleet of VMs

C. On a container

D. Using Terraform

E. On a storage bucket

F. Using Cloud Run functions to trigger the installation

Proprietary + Confidential

Quiz | Question 02

Answer

What are the three ways to install the Ops Agent?

A. On a single VM 

B. On a fleet of VMs 

C. On a container

D. Using Terraform 

E. On a storage bucket

F. Using Cloud Run functions to trigger the installation

Proprietary + Confidential

Quiz | Question 03

Question

What is used to collect metrics inside the VM instead of at the hypervisor level?

A. Ops Agent

B. Cloud Monitoring

C. Monarch

D. Graphana

Proprietary + Confidential

Quiz | Question 03

Answer

What is used to collect metrics inside the VM instead of at the hypervisor level?

A. Ops Agent 

B. Cloud Monitoring

C. Monarch

D. Graphana

Proprietary + Confidential

Recap

01 Use Ops Agent with Compute Engine.

02 Explain the benefits of using Google Cloud Managed Service for Prometheus.

03 Explain the usage of Prometheus Query Language (or PromQL) to query Cloud Monitoring metrics.

04 Explain custom metrics.



Proprietary + Confidential

Reference material

- Install Ops Agent by using Ansible
- Cloud Monitoring metrics in Managed Prometheus

Proprietary + Confidential